

MAC 233: Hall Effect Sensor Lab

Dr Max Champneys

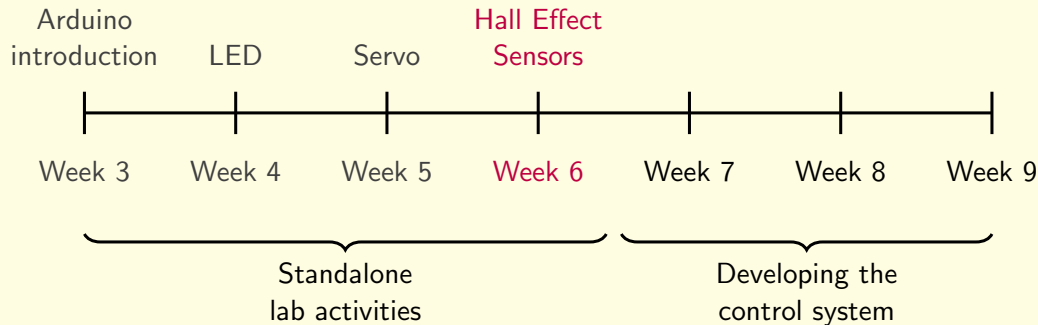
School of Mechanical, Aerospace, and Civil Engineering,
University of Sheffield, UK

MAC 233: **04/11/2026**



University of
Sheffield

MAC 233: LABS OVERVIEW



TODAY'S LAB: PREREQUISITES

This lab doesn't require any code from previous labs; however, it does expect you to have completed these, as common code repeated from previous labs will not be explained in detail again here.

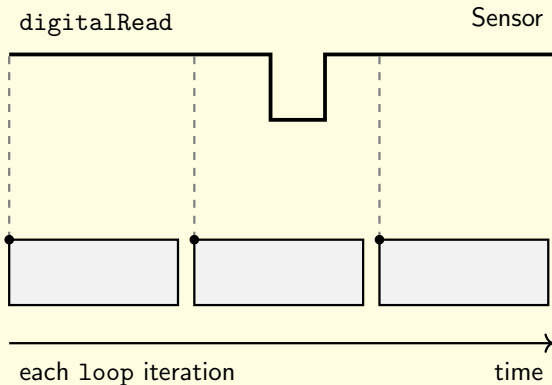
Out of your Arduino pack, you will need: 1 Arduino, 1 breadboard, 1 A3144 hall effect sensor, 1 magnet, 1 LED, 1 $10K\Omega$ resistor, 1 220Ω resistor, and 5 jumper cables.

TODAY'S LAB: OBJECTIVES

The objective of today's lab is to learn how to use an interrupt to record a value of a Hall effect sensor when a magnet passes it.

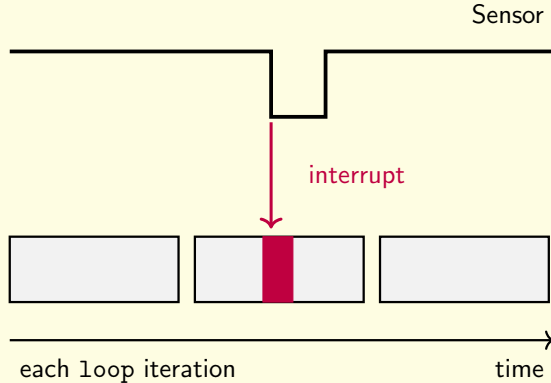
`https://docs.arduino.cc/language-reference/en/functions/external-interrupts/attachInterrupt/`.

READING AN INPUT: WHY WE NEED AN INTERRUPT



An input can change at any moment, but the 'loop' function only looks at it once per iteration. A magnet that comes and goes in between is never seen.

READING AN INPUT: THE INTERRUPT



An interrupt pauses the 'loop' function the moment the pin changes, runs a function of your choosing, and then carries on where it left off.

NEW: 'ATTACHINTERRUPT'

One line in setup says which pin to watch, which function to call, and when:

```
attachInterrupt(digitalPinToInterrupt(CRANK_PIN), crankInterrupt, FALLING);
```

- ▶ `digitalPinToInterrupt(CRANK_PIN)` — the pin to watch
- ▶ `crankInterrupt` — the function to call, with no brackets
- ▶ `FALLING` — when: as the pin drops from 5 V to 0 V

NEW: THE INTERRUPT FUNCTION

The function you named is an ordinary function, written beside `setup` and `loop` — never inside them:

```
void crankInterrupt() {  
    crank_interrupt_current_time = millis();  
}
```

It runs the moment the magnet arrives, whatever the 'loop' function was part-way through. Keep it as short as you can: record what you need, and get out.

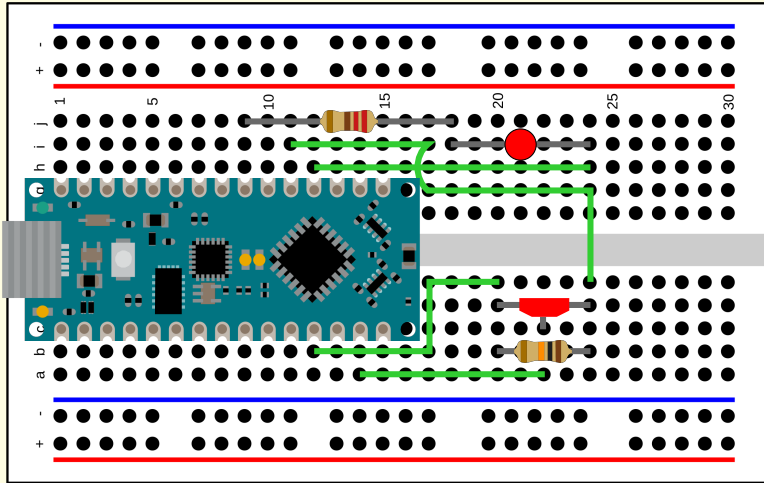
VARIABLES: 'VOLATILE'

An interrupt can change a variable at any moment. Marking it 'volatile' tells the compiler not to cache the value, so the main 'loop' always sees what the interrupt wrote.

```
// written by the interrupt, read by the loop  
volatile unsigned long crank_interrupt_current_time;
```

```
// only ever touched by the loop, so no volatile  
unsigned long current_loop_time;
```

WIRING DIAGRAM

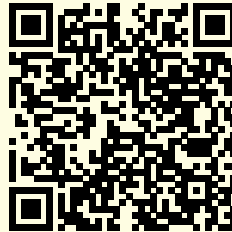


Breadboard source: <https://freesvg.org/breadboard>, Resistor source <https://freesvg.org/resistor-330-ohm>.

Lab 4 worksheet



Nano Every pinout



Problems? Raise your hand and someone will come and help you.